GENERAL

Technical Post:

No Threads Attached: Event-Driven Sleep Obfuscation on Linux

Keiran Mather, - Red Team Specialist 27/06/2025 30 min read

Introduction

CONTENTS

Sleep obfuscation has quickly become a staple capability leveraged by modern malware for evasion tactics. Windows implementations like Ekko and Foliage apply various techniques to hide a process while idle. Sleep obfuscation techniques often leverage Windows-specific primitives such as thread suspend APIs, Asynchronous Procedure Call (APC), and timer callbacks. These are used to execute sleep while encrypting process memory to actively conceal it from memory scanners.

As these capabilities continue to evolve for execution, the defense ecosystem grows in tandem. Endpoint Detection and Response (EDR) solutions that target Windows environments primarily, have begun to rapidly extend their coverage to Unix-like systems, e.g., Linux and MacOS. Modern EDR agents available for Unix-like systems already incorporate many of the same detection capabilities as Windows EDR Agents such as memory scanning, process injection detection, and behavioural analysis.

The expanded coverage of EDRs reflects the reality of a complicated enterprise ecosystem where platform distribution varies significantly based on the organisational context. Enterprises that follow traditional patterns, will primarily stick to Windows management policies where end-users are restricted to comply with prescribed environments and processes. Whereas technology enterprises typically deploy a diverse array of infrastructure, with a large MacOS user population and Linux workstations. Also, whilst there is coverage of EDR for Unix based platforms, the majority only seem to focus on server based infrastructure.

Background: From Windows Timers to Linux Events

Technical Architecture Overview

Implementation Deep Dive: Dissecting the Execution Flow

The Critical Limitation: When Features Become Problematic

Detecting Context-Based Sleep Obfuscation: Stack Analysis During Encrypted Sleep

While there has been some progress in extending the focus on detection coverage across systems, to date, all forms of EDR have been demonstrated significant capability gaps outside the Windows world. However, with the growth in advanced cross-platform attack frameworks and the growing use of Unix systems in higher value enterprise environments, detection capabilities are maturing rapidly across all platforms.

Recent research by Kyle Avery and others have emphasised how EDR detection methodologies are being progressively adapted for Unix. In addition, Linux EDR products regularly and routinely use eBPF-based syscall monitoring, kernel modules monitoring and memory scanning capabilities akin to what is available to Windows EDRs. The days of Unix systems being a "detection-free zone" are quickly vanishing, creating new operational challenges for red team operators, and requiring new platform-specific evasion research.

This research presents, SilentPulse, a *prototype* single-threaded sleep obfuscation capability for Linux-based systems. In its simplest form, SilentPulse uses event-driven architecture utilising Linux's epoll facility, as well as file descriptor-based timers (timerfd) and event notifications (eventfd).

From a technical implementation, the variations in using an event-driven architecture has the potential to provide advantages in both operational flexibility, as well as detection avoidance based on the task design. By treating timers as file descriptors and incorporating them as any other

event in the epoll event loop, a resilient architecture was achieved which not only waits for a timer event but also supports a number of possibilities that are event-based. These include interrupts that are network events or file system changes and therefore are inherently external to the program. The method to accomplish sleep obfuscation can now be viewed as more than a timer mechanism to delay or obfuscate, it now becomes an event-based design capable supporting higher levels of operations.

SHARE THIS ARTICLE

Background: From Windows Timers to Linux Events

To understand SilentPulse we need to look at the existing sleep obfuscation implementations, mainly the Windows specific Ekko implementation and the Linux specific Pendulum implementation. Ekko uses a methodology which can be seen across many different examples, which includes creating a timer, encrypting the executable memory of the process, waiting for the timer to complete and then decrypting the executable memory and continuing execution. This method works very well against Windows EDR solutions that utilise memory scanning. The Pendulum implementation leverages POSIX timers with a SIGEV_THREAD notification, creating a callback function that executes in a separate thread context when the timer expires. This design maintains conceptual similarity to Windows implementations while adapting to Unix system programming models.

```
// Pendulum's core timer mechanism
struct sigevent sev; sev.sigev_notify = SIGEV_THREAD;
sev.sigev_notify_function = timer_callback;
sev.sigev_value.sival_ptr = &context_data;
timer_create(CLOCK_REALTIME, &sev, &timer_id);
```

The fLink's research with
[Hunt-Sleeping-Beacons]
, highlighted detection techniques that look for indicators of compromise by identifying unbacked memory or stomped modules. This enables the detection of multiple proof-of-concepts for sleepmasks that utilise APCs or Timers. This is achieved via analysing the callstack, enumerating timers and their exact callbacks from userland. This detection technique can be similarly tailored to Linux based environments via callstack analysis as well.

Both Pendulum and SilentPulse share a fundamental detection vector: when a process sleeps and memory is encrypted, the call stack still has return addresses pointing to encrypted memory regions that are temporarily marked non-executable. This creates detectable artifacts because legitimate return addresses should always be pointing to executable memory regions.

SilentPulse adopts a different design approach by using event-driven programming that is typically found in modern Linux applications, instead of using threads and signal handlers. SilentPulse handles sleep as an I/O operation in a signaled state, relying on the same mechanisms that high-performance network servers and real-time applications communicate with. Although the approach provides operational benefits, the basic stack-based detection challenges are similar for all implementations, and these detection vectors with their respective mitigation will be explored later.

Technical Architecture Overview

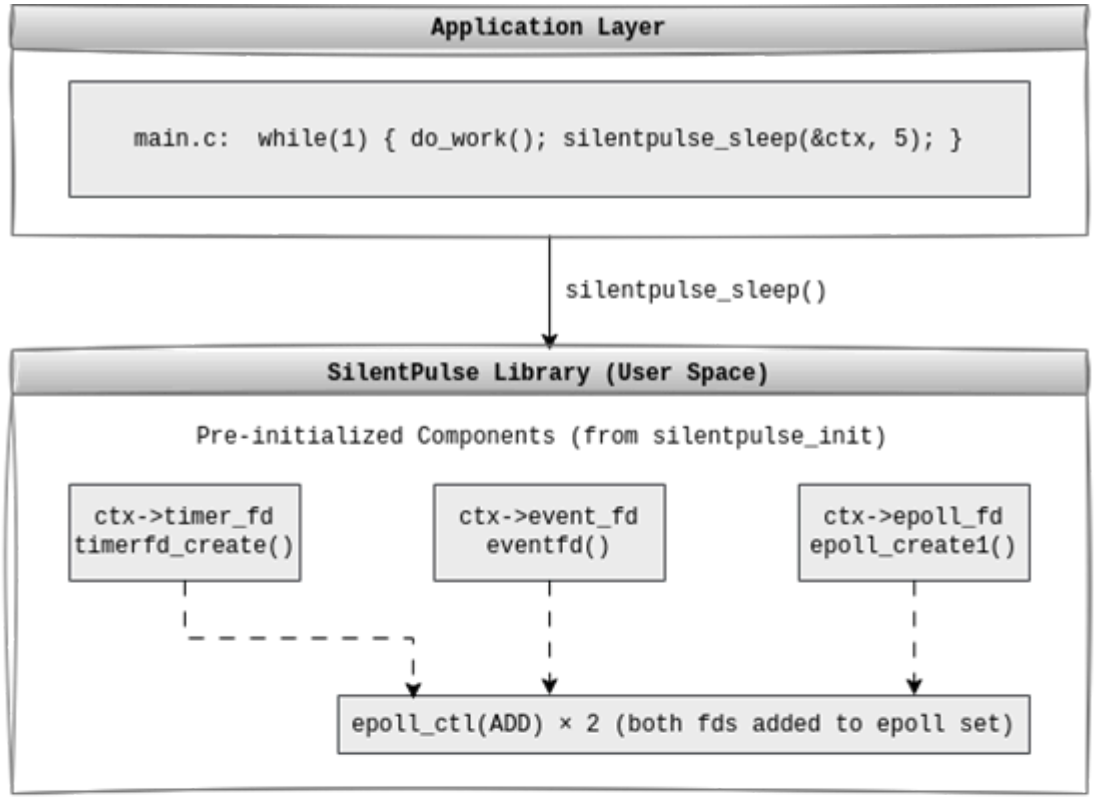
SilentPulse employs an event-driven approach to sleep obfuscation that integrates with Linux's I/O subsystem. The design centers on epoll's event multiplexing capabilities, which provide efficient monitoring of multiple file descriptors simultaneously. Through

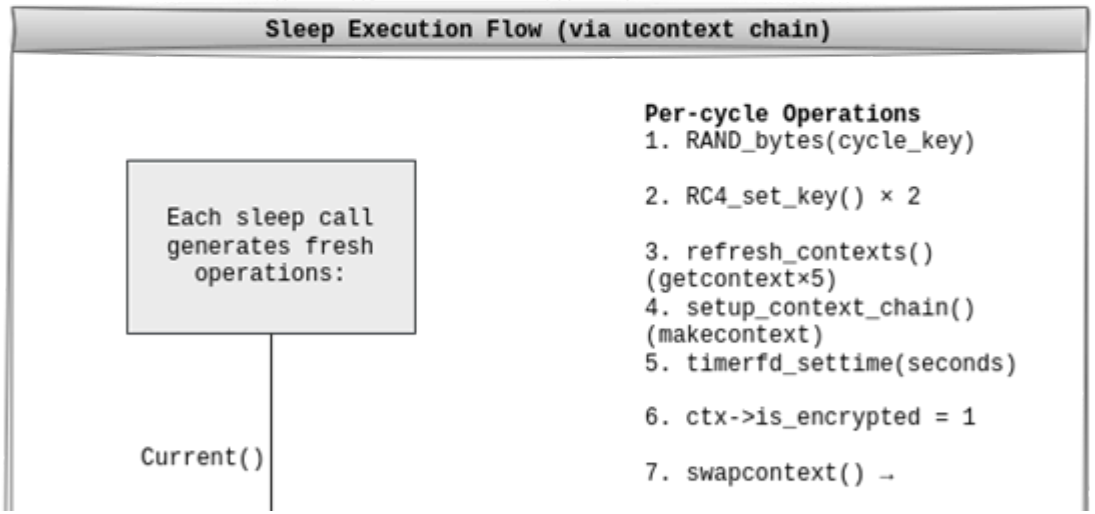
Linux's timerfd interface, sleep timers become file descriptors that integrate directly into the event loop, allowing sleep expiration to be handled alongside other I/O events in a unified processing model.

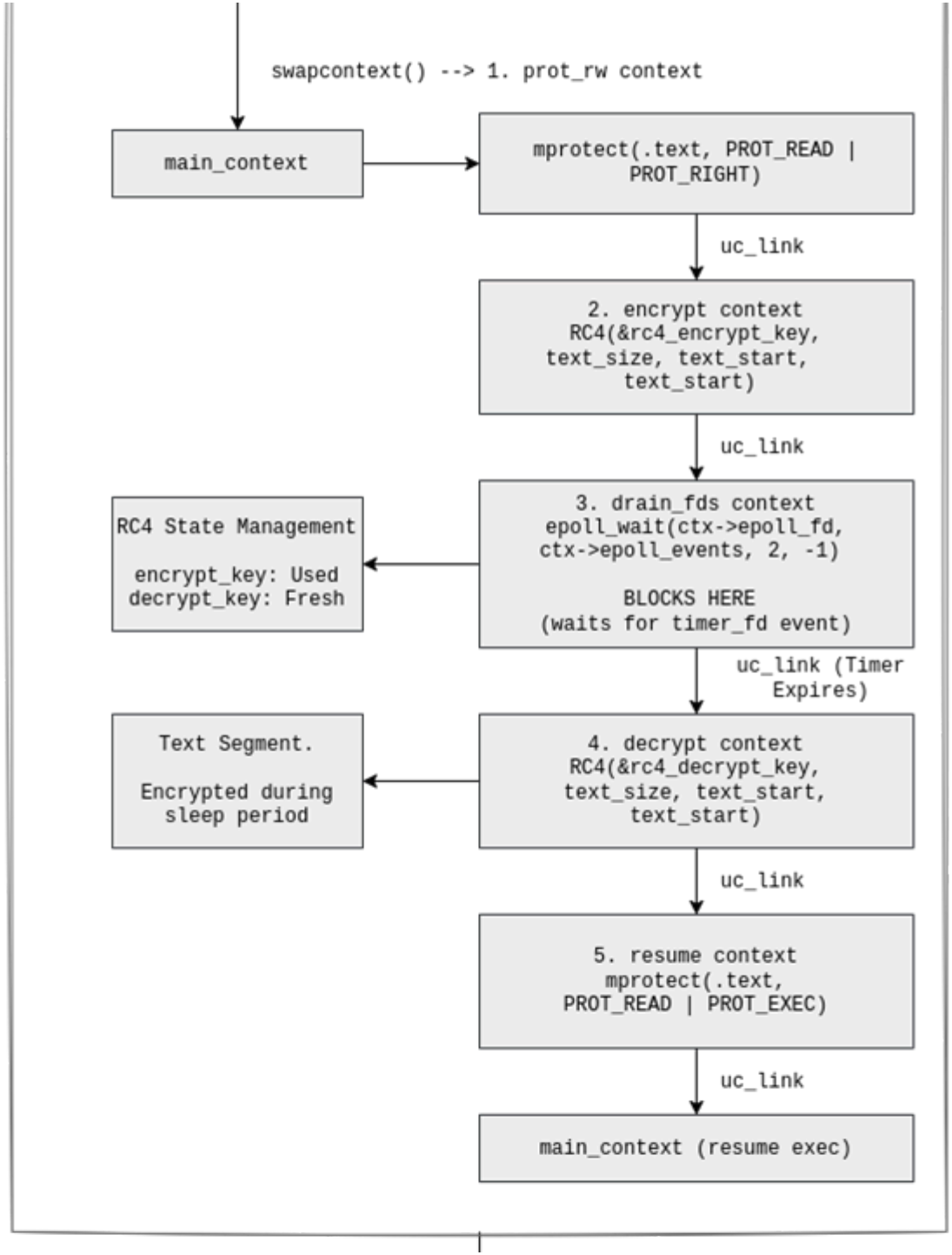
Core Linux Primitives

The implementation is built using three key Linux-system Interfaces:

1. epoll
   - The primary event multiplexing mechanism which allows the ability to monitor multiple file descriptors to see if they are 'ready.' Unlike more conventional methods of determining readiness such as blocking I/O or signal-based methods, epoll can provide an elegant interface for waiting on more than one source of event notifications.
2. timerfd
   - A Linux-specific interface that exposes timers using file descriptors. This means that timers are naturally placed into epoll event loops instead of requiring some other way of managing timed events through signals or threads.
3. eventfd
   - A low-level notification "event" mechanism, which comprises file descriptors whose primary purpose is to notify different parts of a program that an event occurred through either an internal API or even external triggers. While not central to the implementation here, eventfd provides a useful extension point for additional external triggering mechanisms.


Application Layer
main.c: while(1) { do_work(); silentpulse_sleep(&ctx, 5); }
silentpulse_sleep()
SilentPulse Library (User Space)
Pre-initialized Components (from silentpulse_init)
ctx->timer_fd
timerfd_create()
ctx->event_fd
eventfd()
ctx->epoll_fd
epoll_create1()
epoll_ctl(ADD) × 2 (both fds added to epoll set)


Sleep Execution Flow (via ucontext chain)
Each sleep call generates fresh operations:
Current()
Per-cycle Operations
1. RAND_bytes(cycle_key)
2. RC4_set_key() × 2
3. refresh_contexts() (getcontext×5)
4. setup_context_chain() (makecontext)
5. timerfd_settime(seconds)
6. ctx->is_encrypted = 1
7. swapcontext() →


swapcontext() --> 1. prot_rw context
main_context
mprotect(.text, PROT_READ | PROT_RIGHT)
uc_link
2. encrypt context RC4(&rc4_encrypt_key, text_size, text_start, text_start)
uc_link
3. drain_fds context epoll_wait(ctx->epoll_fd, ctx->epoll_events, 2, -1)
BLOCKS HERE (waits for timer_fd event)
RC4 State Management
encrypt_key: Used
decrypt_key: Fresh
uc_link (Timer Expires)
4. decrypt context RC4(&rc4_decrypt_key, text_size, text_start, text_start)
Text Segment.
Encrypted during sleep period
uc_link
5. resume context mprotect(.text, PROT_READ | PROT_EXEC)
uc_link
main_context (resume exec)

Implementation Deep Dive: Dissecting the Execution Flow

Context Chain Mechanics: The Heart of Execution Control

SilentPulse utilises the POSIX ucontext API to make a deterministic chain of execution. Instead of function calls or signal callbacks, this implementation builds a linked chain of execution contexts allowing a defined pathway for the process to take with every step of the sleep cycle. This builds on a few of the concepts from the Pendulum proof-of-concept, but SilentPulse's event-driven integration provides a different architectural approach to the sleep mechanism.

Understanding ucontext Fundamentals

The ucontext API provides three key functions for execution control:

```
getcontext(ucontext_t ucp);                          // Capture current execution state
makecontext(ucontext_t ucp, void (*func)(), int argc, ...);   // Prepare context for function
setcontext(const ucontext_t ucp);                    // Jump to context (no return)
swapcontext(ucontext_t oucp, const ucontext_t *ucp);  // Save current, jump to new
```

SilentPulse leverages these primitives to create a five-stage execution pipeline:

```
typedef struct sleep_contexts {
  ucontext_t prot_rw;      // Stage 1: Memory protection change
  ucontext_t encrypt;      // Stage 2: RC4 encryption
  ucontext_t drain_fds;    // Stage 3: epoll_wait (sleep)
  ucontext_t decrypt;      // Stage 4: RC4 decryption
```

```
    ucontext_t resume;       // Stage 5: Memory protection restore
    ucontext_t main_context; // Return point to application
  } sleep_contexts_t;
```

Context Chain Construction

The implementation establishes execution flow through the
uc_link
field, which specifies the next context to execute when the current function returns:

```
  static void setup_context_chain(silentpulse_ctx* ctx) {
    internal_state_t* state = (internal_state_t*)ctx->sleep_contexts;
    // Stage 1: Change memory protection to RW
    state->ctx.prot_rw.uc_link = &state->ctx.encrypt;
    makecontext(&state->ctx.prot_rw, (void (*)(void))mprotect, 3, ctx->text_start, ctx->text_size, PROT_READ | PROT_WRITE);

    // Stage 2: Encrypt .text segment
    state->ctx.encrypt.uc_link = &state->ctx.drain_fds;
    makecontext(&state->ctx.encrypt, (void (*)(void))RC4, 4, &state->rc4_encrypt_key, ctx->text_size, ctx->text_start, ctx->

    // Stage 3: Sleep via epoll_wait
    state->ctx.drain_fds.uc_link = &state->ctx.decrypt;
    makecontext(&state->ctx.drain_fds, (void (*)(void))epoll_wait, 4, ctx->epoll_fd, ctx->epoll_events, 2, -1);    //

    Stage 4: Decrypt .text segment
    state->ctx.decrypt.uc_link = &state->ctx.resume;
    makecontext(&state->ctx.decrypt, (void (*)(void))RC4, 4, &state->rc4_decrypt_key, ctx->text_size, ctx->text_start, ctx->

    // Stage 5: Restore execute permissions
```

```
  state->ctx.resume.uc_link = &state->ctx.main_context;
  makecontext(&state->ctx.resume, (void (*)(void))mprotect, 3, ctx->text_start, ctx->text_size, PROT_READ | PROT_EXEC);}
```

This chain creates a predictable execution flow where each function automatically transfers control to the next stage upon completion. While the ucontext mechanism eliminates traditional function call overhead and return value propagation, each context transition still requires explicit swapcontext invocations to transfer execution control between phases.

Memory Protection and .text Segment Discovery

The implementation uses linker symbols to precisely locate the executable code segment:

```
extern char __executable_start;
extern char __etext;

static int discover_text_segment(silentpulse_ctx* ctx) {
  ctx->text_start = &__executable_start;
  ctx->text_size = (size_t)&__etext - (size_t)&__executable_start;
  return 1;
}
```

These symbols, provided by the GNU linker, mark the exact boundaries of the executable code section. The linker script defines these boundaries explicitly, as demonstrated in the output from
ld –verbose:

```
SECTIONS
{
 PROVIDE (__executable_start = SEGMENT_START("text-segment", 0x400000));
 . = SEGMENT_START("text-segment", 0x400000) + SIZEOF_HEADERS;

---snip---
```

This reveals how the linker establishes the
__executable_start
symbol at the beginning of the text segment, typically at virtual address 0x400000 for x86_64 binaries. This approach ensures precision in targeting only the code that needs protection, avoiding data segments or other memory regions. However, this technique can be extended to include heap encryption for broader memory protection coverage.

RC4 Encryption and Event System Integration

SilentPulse generates a fresh RC4 key for each sleep cycle, enhancing security by ensuring that encrypted memory contents vary between iterations:

```
int silentpulse_sleep(silentpulse_ctx* ctx, int seconds) {
  unsigned char cycle_key[RC4_KEY_SIZE];

  // Generate fresh key for this cycle
  if (RAND_bytes(cycle_key, RC4_KEY_SIZE) != 1) {
      fprintf(stderr, "failed to generate random RC4 key for cycle\n");
      return 0;
   }

   // Initialize separate encrypt/decrypt key structures
   RC4_set_key(&state->rc4_encrypt_key, RC4_KEY_SIZE, cycle_key);
   RC4_set_key(&state->rc4_decrypt_key, RC4_KEY_SIZE, cycle_key);
}
```

The implementation leverages Linux's timerfd interface to create file descriptor-based timers:

```
static int setup_event_system(silentpulse_ctx* ctx) {
   // Create monotonic timer file descriptor
   ctx->timer_fd = timerfd_create(CLOCK_MONOTONIC, TFD_CLOEXEC);

   // Create event notification file descriptor
```

```
    ctx->event_fd = eventfd(0, EFD_CLOEXEC);

    // Create epoll instance for event multiplexing
    ctx->epoll_fd = epoll_create1(EPOLL_CLOEXEC);

    // Register timer with epoll
    struct epoll_event ev = {.events = EPOLLIN, .data.fd = ctx->timer_fd};
    epoll_ctl(ctx->epoll_fd, EPOLL_CTL_ADD, ctx->timer_fd, &ev);
}
```

Complete Execution Flow: Detailed System-Level Analysis

Initiation: Context State Preservation and Transfer

The sleep cycle initiation involves a key
swapcontext()
operation that performs atomic state preservation and control transfer:

```
swapcontext(&state->ctx.main_context, &state->ctx.prot_rw);
```

At a low level,
swapcontext()
executes a sequence of userspace operations that preserve the user-visible CPU state. On x86-64 architectures, this encompasses saving all general-purpose registers (RAX through R15), the instruction pointer (RIP), stack pointer (RSP), base pointer (RBP), and the RFLAGS register into the
main_context
structure. The implementation, typically residing in libc as optimised assembly code, then performs the reciprocal operation, loading the previously saved register state from
prot_rw

, thereby
seamlessly
transferring execution to the target context.

The main_context structure now encapsulates the complete execution state necessary for seamless resumption after the sleep cycle completes. This includes the precise instruction pointer location following the swapcontext() call, the intact stack frame configuration, and all register values required to continue execution as if no context switch had occurred.

Memory Protection Modification: Kernel VMA Manipulation

The first stage executes mprotect() to modify the Virtual Memory Area (VMA) attributes of the .text segment:

```
mprotect(ctx->text_start, ctx->text_size, PROT_READ | PROT_WRITE);
```

This system call triggers a complex kernel operation within the memory management subsystem. The kernel locates the VMA structure corresponding to the .text segment through the process's VMA maple tree (mm→mm_mt in the process's mm_struct), as implemented in the kernel's
find_vma
function which performs efficient VMA lookup.

For the given address range, the kernel must also:

1. VMA Structure Update: Modify the vm_flags field to include VM_READ | VM_WRITE and remove VM_EXEC. The vm_page_prot field is recalculated based on these new flags.

2. Page Table Traversal: Walk the multi-level page table hierarchy (PGD→PUD→PMD→PTE on x86-64) to locate each page table entry within the specified range, updating the protection bits in each PTE.

3. TLB Invalidation: Issue TLB shootdown operations via flush_tlb_range() to invalidate cached address translations on all CPUs where the process might have run, ensuring the protection changes take immediate effect.

The change in protection is significant because writing to read-only pages would cause a page fault exception handled by the kernel's fault handler. The MMU enforces these permissions by checking the page table entry's protection bits against the

requested access type on every memory reference. Once it's done, mprotect() resumes the ucontext mechanism, which automatically transfers control to the encryption stage via the uc_link pointer established during our context chain setup.

Code Encryption: In-Place Cryptographic Transformation

The encryption stage performs direct manipulation of the process's executable code using the OpenSSL RC4 implementation:

```
RC4(&state->rc4_encrypt_key, ctx->text_size, ctx->text_start, ctx->text_start);
```

This operation modifies the binary representation for that process's executable code. More importantly, the source and destination pointers in this operation refer to the same memory address (ctx→text_start), thus, in-place modification is possible.

At the byte level, this process transforms recognisable x86-64 instruction sequences into cryptographically scrambled data through XOR operations. For example, a common function prologue gets transformed as follows:

```
Original instruction: 55 48 89 e5        (push %rbp; mov %rsp,%rbp)
RC4 keystream:        f6 ba 17 f9        (pseudorandom bytes from RC4)
XOR result:           a3 f2 9e 1c        (55xf6, 48xba, 89x17, e5xf9)
```

The encrypted bytes a3 f2 9e 1c no longer represent valid x86-64 instructions, appearing as random data.

The RC4 key structure maintains internal state including the S-box permutation and the i,j counters that determine keystream generation. This state is crucial because identically initialised key structures will produce identical keystream output. Since RC4 operations modify the key structure's internal state, SilentPulse maintains separate encrypt and decrypt key structures, both initialised with the same key material, ensuring the decryption process generates the exact same keystream sequence as encryption, enabling perfect reversal of the encryption process.

Event Loop Sleep: Kernel Event Multiplexing

The sleep stage represents the core blocking operation where the process enters a dormant state:

```
epoll_wait(ctx->epoll_fd, ctx->epoll_events, 2, -1);
```

This system call transfers the control flow to the kernel's epoll implementation, which does multiple important things:

- Validation of the File Descriptors: The kernel checks that ctx→epoll_fd is a valid epoll instance and checks that the registered file descriptors (timerfd and eventfd) have been set up correctly.
- Wait Queue Management: The current process is added to the wait queues associated with each monitored file descriptor. For the timerfd, this means the process becomes a waiter on the timer's expiration event.
- Scheduler Interaction: The process state transitions from TASK_RUNNING to TASK_INTERRUPTIBLE, removing it from the scheduler's active run queue. The process will only be rescheduled when one of the monitored events occurs or when interrupted by a signal.
- Memory State: Notably, during this sleep period, the .text segment remains encrypted. Any external memory scanning or process inspection will encounter the RC4-encrypted data rather than the original executable code.

The timeout parameter of -1 indicates an indefinite wait, meaning the process will only wake when the timerfd expires (when our sleep duration elapses) or when interrupted by a signal such as those generated by debugging tools.

Code Decryption: Cryptographic State Restoration

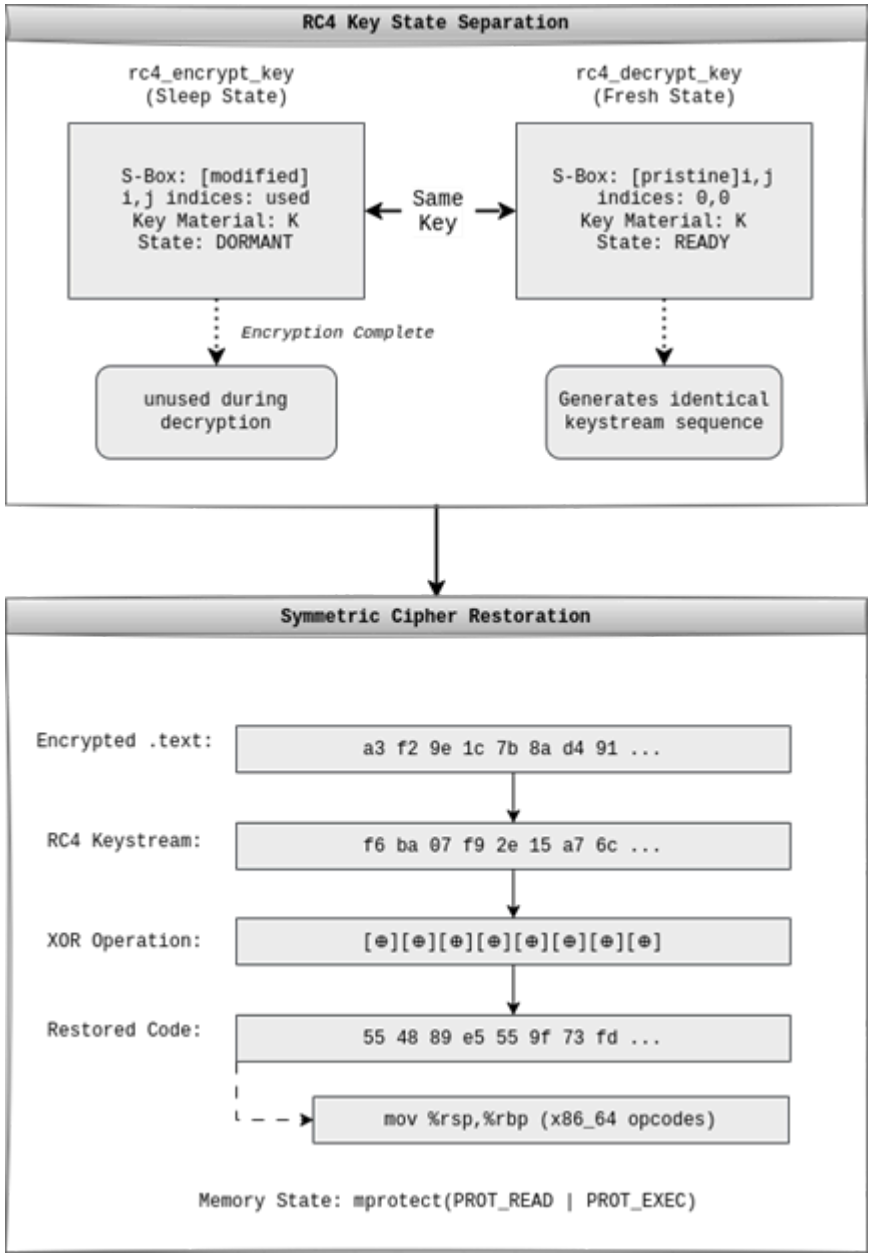
Upon wakeup, the decryption stage reverses the encryption transformation:

```
RC4(&state->rc4_decrypt_key, ctx->text_size, ctx->text_start, ctx->text_start);
```

The essential implementation detail is that both
rc4_encrypt_key
and
rc4_decrypt_key

were initialised with identical key material but maintain separate RC4 state structures. This separation ensures that the decryption operation begins with a fresh RC4 state, generating the same keystream sequence that was used during encryption.

Since RC4 is a symmetric stream cipher where encryption and decryption are identical operations (XOR with the same keystream), this process restores the original executable code:



The restoration occurs byte-by-byte across the entire .text segment, ensuring that everything is returned to its original state.

Permission Restoration: VMA Security Enforcement

The final stage restores the .text segment's execute permissions:

```
mprotect(ctx->text_start, ctx->text_size, PROT_READ | PROT_EXEC);
```

This second mprotect() call reverses the protection changes made in the first stage. The kernel again modifies the VMA structure and page table entries, but this time removing write permissions and restoring execute permissions.

After mprotect() completion, the ucontext framework transfers control back to main_context, restoring the application's execution state exactly as it existed before the sleep cycle began. The application continues execution with no awareness that its code was temporarily encrypted and its execution context was manipulated through the kernel's VMA subsystem.

Context Refresh Mechanism

The ucontext structures require refresh because swapcontext() overwrites the first argument (the save context) with the current execution state. While the makecontext()-configured execution contexts can be reused indefinitely, the main_context structure gets overwritten during each swapcontext() call and must be reinitialised with getcontext() before it can serve as a save point again. This refresh pattern enables the same execution chain to be reused across multiple sleep cycles.

```
static int refresh_contexts(internal_state_t* state) {
   ucontext_t* contexts[] = {&state->ctx.prot_rw, &state->ctx.encrypt,
                             &state->ctx.drain_fds, &state->ctx.decrypt,
                             &state->ctx.resume};
   for (size_t i = 0; i < sizeof(contexts) / sizeof(contexts[0]); i++) {
      if (getcontext(contexts[i]) == -1) {
          perror("getcontext");
          return 0;
        }
   }
```

```
    return 1;
}
```

Without refresh, subsequent sleep cycles would fail because swapcontext() overwrites the main_context save point with the current execution state. The getcontext() call reinitialises each context to a clean state, after which makecontext() reconfigures the execution chain in setup_context_chain(). This design pattern allows repeated execution of the same mprotect→encrypt->wait→decrypt→mprotect sequence across multiple sleep cycles.

Memory State Visualisation

The transformation process creates distinct memory states throughout execution, as demonstrated by the following memory dumps from a running SilentPulse process:

Initial State - ELF Header and String Table (Awake):

```
00000000  7f 45 4c 46 02 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
00000010  03 00 3e 00 01 00 00 00  a0 11 00 00 00 00 00 00  |..>.............|
00000020  40 00 00 00 00 00 00 00  f0 3b 00 00 00 00 00 00  |@........;......|
00000030  00 00 00 00 40 00 38 00  0e 00 40 00 1e 00 1d 00  |....@.8...@.....|
00000040  06 00 00 00 04 00 00 00  40 00 00 00 00 00 00 00  |........@.......|
00000050  40 00 00 00 00 00 00 00  40 00 00 00 00 00 00 00  |@.......@.......|
00000060  10 03 00 00 00 00 00 00  10 03 00 00 00 00 00 00  |................|
00000070  08 00 00 00 00 00 00 00  03 00 00 00 04 00 00 00  |................|
---snip---
000006f0  08 00 00 00 00 00 00 00  00 5f 49 54 4d 5f 64 65  |........._ITM_de|
00000700  72 65 67 69 73 74 65 72  54 4d 43 6c 6f 6e 65 54  |registerTMCloneT|
00000710  61 62 6c 65 00 5f 5f 67  6d 6f 6e 5f 73 74 61 72  |able.__gmon_star|
00000720  74 5f 5f 00 5f 49 54 4d  5f 72 65 67 69 73 74 65  |t__._ITM_registe|
00000730  72 54 4d 43 6c 6f 6e 65  54 61 62 6c 65 00 52 41  |rTMCloneTable.RA|
00000740  4e 44 5f 62 79 74 65 73  00 52 43 34 5f 73 65 74  |ND_bytes.RC4_set|
00000750  5f 6b 65 79 00 52 43 34  00 65 70 6f 6c 6c 5f 63  |_key.RC4.epoll_c|
```

```
00000760  74 6c 00 73 6c 65 65 70  00 70 75 74 73 00 70 65  |tl.sleep.puts.pe|
00000770  72 72 6f 72 00 73 79 73  63 6f 6e 66 00 5f 5f 73  |rror.sysconf.__s|
00000780  74 61 63 6b 5f 63 68 6b  5f 66 61 69 6c 00 66 72  |tack_chk_fail.fr|
00000790  65 65 00 74 69 6d 65 72  66 64 5f 63 72 65 61 74  |ee.timerfd_creat|
000007a0  65 00 72 65 61 64 00 74  69 6d 65 72 66 64 5f 73  |e.read.timerfd_s|
000007b0  65 74 74 69 6d 65 00 6d  61 6b 65 63 6f 6e 74 65  |ettime.makeconte|
000007c0  78 74 00 67 65 74 70 69  64 00 67 65 74 63 6f 6e  |xt.getpid.getcon|
000007d0  74 65 78 74 00 5f 5f 6c  69 62 63 5f 73 74 61 72  |text.__libc_star|
000007e0  74 5f 6d 61 69 6e 00 73  74 64 65 72 72 00 6d 70  |t_main.stderr.mp|
000007f0  72 6f 74 65 63 74 00 73  77 61 70 63 6f 6e 74 65  |rotect.swapconte|
00000800  78 74 00 65 76 65 6e 74  66 64 00 5f 5f 63 78 61  |xt.eventfd.__cxa|
00000810  5f 66 69 6e 61 6c 69 7a  65 00 65 70 6f 6c 6c 5f  |_finalize.epoll_|
00000820  63 72 65 61 74 65 31 00  63 61 6c 6c 6f 63 00 6d  |create1.calloc.m|
00000830  65 6d 73 65 74 00 63 6c  6f 73 65 00 70 72 69 6e  |emset.close.prin|
00000840  74 66 00 66 77 72 69 74  65 00 6c 69 62 73 73 6c  |tf.fwrite.libssl|
00000850  2e 73 6f 2e 33 00 6c 69  62 63 72 79 70 74 6f 2e  |.so.3.libcrypto.|
00000860  73 6f 2e 33 00 6c 69 62  63 2e 73 6f 2e 36 00 4f  |so.3.libc.so.6.O|
00000870  50 45 4e 53 53 4c 5f 33  2e 30 2e 30 00 47 4c 49  |PENSSL_3.0.0.GLI|
00000880  42 43 5f 32 2e 33 2e 32  00 47 4c 49 42 43 5f 32  |BC_2.3.2.GLIBC_2|
00000890  2e 39 00 47 4c 49 42 43  5f 32 2e 37 00 47 4c 49  |.9.GLIBC_2.7.GLI|
000008a0  42 43 5f 32 2e 34 00 47  4c 49 42 43 5f 32 2e 38  |BC_2.4.GLIBC_2.8|
000008b0  00 47 4c 49 42 43 5f 32  2e 33 34 00 47 4c 49 42  |.GLIBC_2.34.GLIB|
000008c0  43 5f 32 2e 32 2e 35 00  00 00 02 00 02 00 02 00  |C_2.2.5.........|
---snip---
```

Protection: PROT_READ | PROT_EXEC - Clear ELF headers, readable function names, and string table entries

During Sleep - ELF Header and String Table (Encrypted):

```
00000000  14 4b d6 09 ee 31 95 25  cf 67 38 40 ae e7 f8 f6  |.K...1.%.g8@....|
00000010  5b 47 e6 2f 44 b2 de 54  02 de 5c c9 bd 2b 79 24  |[G./D..T..\..+y$|
```

```
00000020  6e 53 21 fb fa 21 77 03  50 76 fe cf 20 0f ba 16  |nS!..!w.Pv.. ...|
00000030  b8 3b 10 3e 5d 15 f9 02  a2 e4 62 63 e3 a7 02 33  |.;.>].....bc...3|
00000040  eb 3d 02 1d ad d2 37 e2  c3 b0 5f 74 f8 7a 95 6e  |.=....7..._t.z.n|
00000050  00 c5 36 0d b8 3b 44 1c  71 cd 80 5f 2e 37 1d 5a  |..6..;D.q.._.7.Z|
00000060  62 ea d1 8f 44 9e cd 0f  74 91 94 1d 17 50 57 ad  |b...D...t....PW.|
00000070  42 bc 85 bd 12 59 42 be  f3 47 06 eb bd 26 ec 76  |B....YB..G...&.v|
---snip---
000006f0  81 63 7c 08 ed 7e c7 34  fa d3 49 ef 2b 35 41 ab  |.c|..~.4..I.+5A.|
00000700  5d 42 e5 96 7d a9 ef 91  b0 27 f4 ad 06 0f b2 52  |]B..}....'.....R|
00000710  f8 82 84 19 14 f9 28 41  25 76 b6 6e 47 9c 43 26  |......(A%v.nG.C&|
00000720  8f 88 80 b9 25 3f 87 c1  5c 57 c1 06 6d 5f 1f a4  |....%?..\W..m_..|
00000730  d6 18 26 ed 49 22 b6 c3  4a e9 07 72 a8 02 77 63  |..&.I"..J..r..wc|
00000740  d4 74 d6 00 43 58 4b 48  50 e5 b9 2c f0 a8 3a 46  |.t..CXKHP..,..:F|
00000750  09 8b 7a e5 f7 3d 27 9c  c1 0b 7e 9d cc e5 7a 1b  |..z..='...~...z.|
00000760  c4 34 a5 7c 2a fe f1 0a  a0 6f 7a d5 d3 b2 d4 7d  |.4.|*....oz....}|
00000770  5f ae 76 61 b0 e1 f2 14  1a c3 1c 7b 90 7c 45 95  |_.va.......{.|E.|
00000780  36 4e c7 5c a0 71 88 ce  39 26 92 96 75 90 3a 29  |6N.\.q..9&..u.:)|
---snip---
```

Protection: PROT_READ | PROT_WRITE

-

Encrypted data, high entropy, no recognisable patterns

Post-Decryption - Fully Restored (Awake):

```
00000000  7f 45 4c 46 02 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
00000010  03 00 3e 00 01 00 00 00  a0 11 00 00 00 00 00 00  |..>.............|
00000020  40 00 00 00 00 00 00 00  f0 3b 00 00 00 00 00 00  |@........;......|
00000030  00 00 00 00 40 00 38 00  0e 00 40 00 1e 00 1d 00  |....@.8...@.....|
00000040  06 00 00 00 04 00 00 00  40 00 00 00 00 00 00 00  |........@.......|
00000050  40 00 00 00 00 00 00 00  40 00 00 00 00 00 00 00  |@.......@.......|
```

```
00000060  10 03 00 00 00 00 00 00  10 03 00 00 00 00 00 00  |................|
00000070  08 00 00 00 00 00 00 00  03 00 00 00 04 00 00 00  |................|
---snip---
000006f0  08 00 00 00 00 00 00 00  00 5f 49 54 4d 5f 64 65  |........._ITM_de|
00000700  72 65 67 69 73 74 65 72  54 4d 43 6c 6f 6e 65 54  |registerTMCloneT|
00000710  61 62 6c 65 00 5f 5f 67  6d 6f 6e 5f 73 74 61 72  |able.__gmon_star|
00000720  74 5f 5f 00 5f 49 54 4d  5f 72 65 67 69 73 74 65  |t__._ITM_registe|
---snip---
```

Protection: PROT_READ | PROT_EXEC

-

Identical to original state, execution can resume normally

The Critical Limitation: When Features Become Problematic

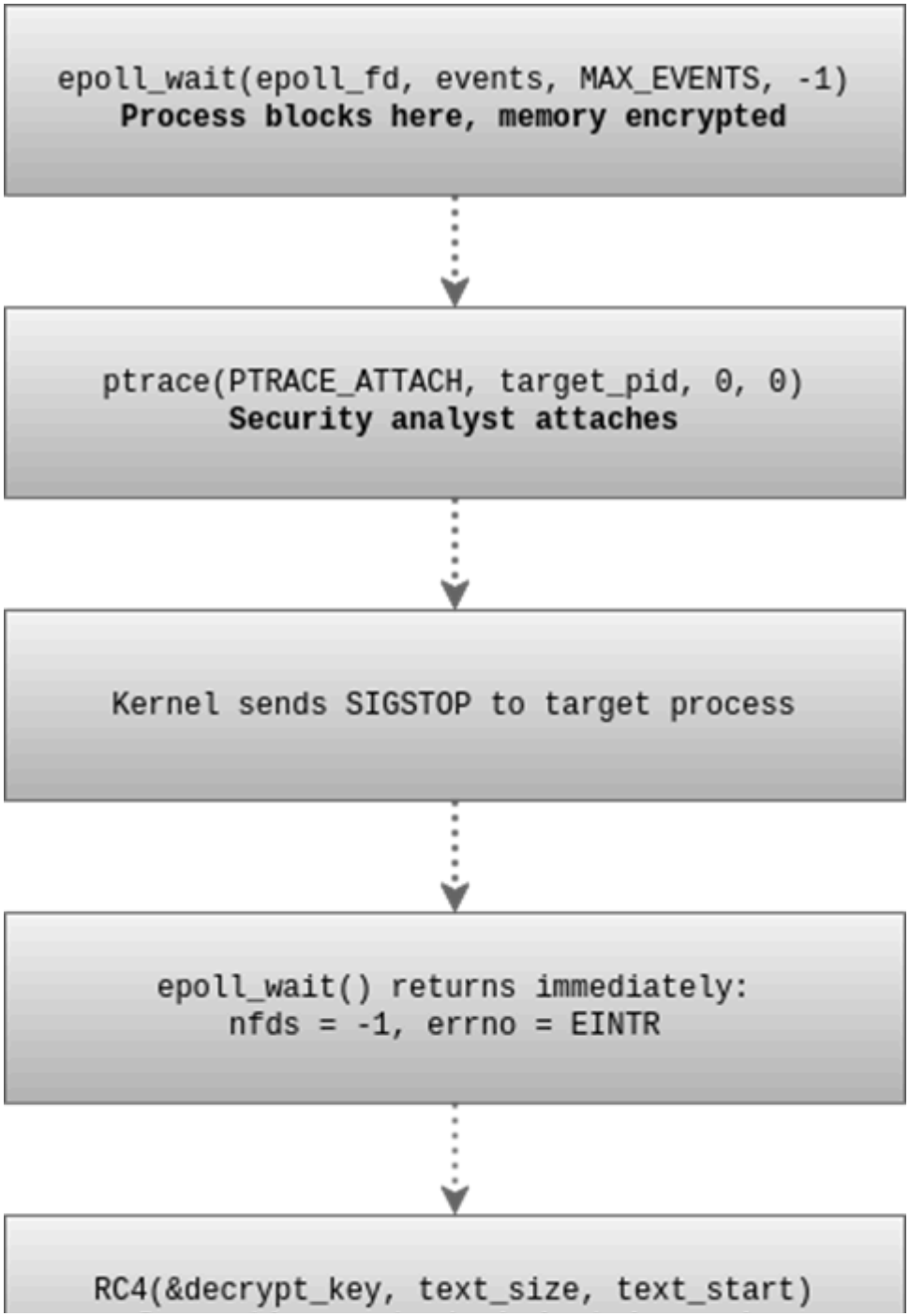
The epoll_wait() Interruption Problem

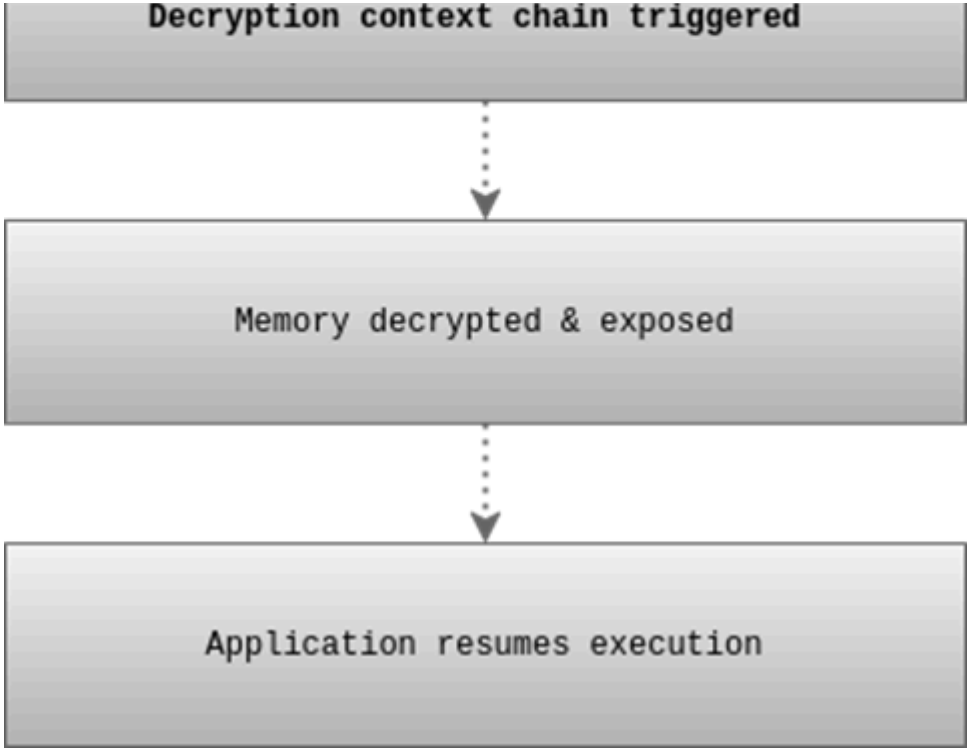
The primary restriction upon the implementation of "SilentPulse" is not on its implementation, but a characteristic of Unix system design which cannot be avoided. SilentPulse uses epoll_wait() as its primary blocking mechanism. Therefore, it consequently suffers from an unavoidable problem based on a longstanding Unix signal processing convention. When epoll_wait() is interrupted by certain signals (e.g., SIGINT), such as the ones created by debugging tools, it will return immediately with errno set to EINTR (Interrupted system call). The operation of immediately returning is not a 'bug' or 'oversight' in the Linux kernel, but rather a feature that is built into the system to maintain process responsiveness towards critical OS events as well as to debugging interfaces. This is by design so you can interrupt these
system calls
in the first place, which is documented both in the official Linux manual pages and in POSIX. The

manual pages
for signal(7) explicitly lists epoll_wait() as a system call that is never restarted after receiving an interrupt from a signal handler (irrespective of SA_RESTART); they always fail with the error EINTR when interrupted by a signal handler. The reasoning behind enabling this interrupt was assuming that a signal happened and the process caught it, then there is a good chance something occurred that should wake up the blocked system call. Also, debugging tools depend on this behavior, specifically while being traced the tracee will stop each time a signal is delivered, even if that signal is being ignored as shown in the
ptrace man page
, as required by debuggers like gdb, strace, and ltrace to function properly. Alternatively, automatic system call restarts would create scenarios where the event loop is stuck in recv() and has no chance to evaluate the flag and exit the program normally, which could break fundamental process control mechanisms such as Ctrl+C handling.

The Anatomy of Interruption

Consider what happens when a security analyst or debugging tool attempts to attach to a sleeping SilentPulse process:


epoll_wait(epoll_fd, events, MAX_EVENTS, -1)
Process blocks here, memory encrypted
ptrace(PTRACE_ATTACH, target_pid, 0, 0)
Security analyst attaches
Kernel sends SIGSTOP to target process
epoll_wait() returns immediately:
nfds = -1, errno = EINTR
RC4(&decrypt_key, text_size, text_start)



The fundamental issue is timing and atomic operations. Once
epoll_wait()
returns due to interruption, the process has already begun its wake-up sequence. The context chain that handles decryption cannot distinguish between a legitimate timer expiration and a malicious interruption, both scenarios trigger the same execution path.

Why This Cannot Be "Fixed"

Understanding why this limitation persists requires examining the fundamental design principles of Unix systems. The interruptibility of system calls like epoll_wait() serves numerous purposes:

1. Process Responsiveness: Unix systems prioritize the ability to interrupt and control processes. This responsiveness is essential for system administration, debugging, and emergency termination of runaway processes.
2. Debugging

Utilities
: Tools like GDB, strace, and other debugging utilities rely on the ability to send signals that interrupt system calls. Without this capability, debugging would be severely limited.

3. Signal Delivery Guarantees: POSIX standards mandate that certain signals must be delivered promptly, which requires interrupting blocking system calls.

```
// This is not a workaround - it's the fundamental issue
while (true) {
    int result = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);

    if (result  -1 && errno  EINTR) {
        // We cannot simply retry here because:
        // 1. The interruption may be malicious (ptrace attach)
        // 2. We are already in the decryption context chain
        // 3. Memory protection has been modified, and .text is decrypted
        // 4. No way to differentiate legitimate vs malicious interruption
        continue;
    }
      // Handle normal events...
}
```

Implications in Practice

This limitation creates a fundamental asymmetry between attacker and defender capabilities. A defender can trivially force decryption with standard debugging tools. Each of these common debugging actions will send signals that interrupt epoll_wait()
, forcing the process to wake up and decrypt its memory. The technique provides no defence against such basic interactions.

```
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

int main(int argc, char *argv[]) {
    pid_t target_pid;
    int status;
    int attempts = 0;

    target_pid = atoi(argv[1]);

    while (1) {
        attempts++;

        if (ptrace(PTRACE_ATTACH, target_pid, NULL, NULL) == -1) {
            usleep(1000);
            continue;
        }

        if (waitpid(target_pid, &status, 0) == -1) {
            ptrace(PTRACE_DETACH, target_pid, NULL, NULL);
            usleep(1000);
            continue;
        }

        if (!WIFSTOPPED(status)) {
            ptrace(PTRACE_DETACH, target_pid, NULL, NULL);
```

```
            usleep(1000);
            continue;
        }

        char maps_path[256];
        snprintf(maps_path, sizeof(maps_path), "/proc/%d/maps", target_pid);
        FILE *maps_file = fopen(maps_path, "r");
        if (maps_file) {
            char line[256];
            unsigned long start_addr = 0;
            if (fgets(line, sizeof(line), maps_file)) {
                sscanf(line, "%lx", &start_addr);
            }
            fclose(maps_file);

            if (start_addr) {
                long exec_data = ptrace(PTRACE_PEEKTEXT, target_pid, start_addr, NULL);
                if (exec_data != -1 || errno == 0) {
                    char *bytes = (char*)&exec_data;
                    if (bytes[0] == 0x7f && bytes[1] == 'E' && bytes[2] == 'L' && bytes[3] == 'F') {
                        printf("caught decrypted after %d attempts! elf header found:\n", attempts);
                        for (int i = 0; i < 16; i++) {
                            long data = ptrace(PTRACE_PEEKTEXT, target_pid, start_addr + i * 8, NULL);
                            if (data != -1 || errno == 0) {
                                printf("__executable_start+%d: 0x%016lx ", i * 8, data);
                                char *dbytes = (char*)&data;
                                for (int j = 0; j < 8; j++) {
                                    if (dbytes[j] >= 32 && dbytes[j] <= 126) {
                                        printf("%c", dbytes[j]);
                                    } else {
                                        printf(".");
                                    }
                                }
                                printf("\n");
```

```
                            }
                        }
                        ptrace(PTRACE_DETACH, target_pid, NULL, NULL);
                        return 0;
                    }
                }
            }
        }

        ptrace(PTRACE_DETACH, target_pid, NULL, NULL);
        usleep(100);
    }

    return 0;
}
```

Example output demonstrates the limitation in practice (of course it would be easier to use gdb and attach, in order to get it in a decrypted state, however, for poc purposes...):

```
$ ./scan 143692

caught decrypted after 1122 attempts! elf header found:
__executable_start+0: 0x00010102464c457f .ELF....
__executable_start+8: 0x0000000000000000 ........
__executable_start+16: 0x00000001003e0003 ..>.....
__executable_start+24: 0x0000000000001190 ........
__executable_start+32: 0x0000000000000040 @.......
__executable_start+40: 0x0000000000003bc0 .;......
__executable_start+48: 0x0038004000000000 ....@.8.
__executable_start+56: 0x001d001e0040000e ..@.....
__executable_start+64: 0x0000000400000006 ........
__executable_start+72: 0x0000000000000040 @.......
__executable_start+80: 0x0000000000000040 @.......
```

```
__executable_start+88: 0x0000000000000040 @.......
__executable_start+96: 0x0000000000000310 ........
__executable_start+104: 0x0000000000000310 ........
__executable_start+112: 0x0000000000000008 ........
__executable_start+120: 0x0000000000000003 ........
```

The SilentPulse process shows corresponding rapid wake cycles:

```
$ ./silentpulse

(0) im alive @ 01:11:19
(1) im alive @ 01:11:24
(2) im alive @ 01:11:27
(3) im alive @ 01:11:27
(4) im alive @ 01:11:27
(5) im alive @ 01:11:27
(6) im alive @ 01:11:27
(7) im alive @ 01:11:27
(8) im alive @ 01:11:27
---snip---
(1118) im alive @ 01:11:28
(1119) im alive @ 01:11:28
(1120) im alive @ 01:11:28
(1121) im alive @ 01:11:28
(1122) im alive @ 01:11:28 <---- caught at this iteration
```

While it took 1,122 attempts to be able to catch the payload in a decrypted state, the demonstration shows that you can indeed overcome this limitation. In reality, SilentPulse still beats automated EDR memory scans that don't utilise ptrace, however, it would still be possible for a security analyst to gather the plaintext with hands-on memory inspection.

When the Limitation Matters

It's important to contextualise when this limitation becomes relevant in practice:

- Memory Scanning

:

Against automated EDR memory scanning that doesn't actively interact with processes, SilentPulse works effectively. Many EDR solutions scan process memory periodically but don't actively attach debuggers/PTRACE_ATTACH to every process.

- Active Investigation

:

Once a process becomes the subject of active investigation by security analysts using debugging tools, the technique's protection can easily be circumvented. However, the same can be said for any technique.

- Research Context

:

For proof-of-concept and educational purposes, the limitation provides valuable insights into the fundamental constraints of this technique, with hopes of inspiring further research in the Linux space.

Detecting Context-Based Sleep Obfuscation: Stack Analysis During Encrypted Sleep

Whilst SilentPulse presents an innovative approach to sleep obfuscation, the implementation exhibits a fundamental limitation shared by other context-switching techniques: the unavoidable forensic artifacts generated by the underlying makecontext()
/
ucontext
execution framework.

The core issue is during the encrypted sleep phase, where return addresses within the preserved execution contexts continue to reference memory regions within the
.text
section - regions that have been marked as non-executable. This creates a detectable anomaly. Execution contexts pointing to non-executable memory ranges, provide a clear indicator of compromise for memory analysis tools.

The Detection Approach: Stack Unwinding Analysis

The detection technique utilises libunwind alongside ptrace to conduct remote stack unwinding of the target process. This methodology reveals a significant insight. Whilst the code remains encrypted, the call stack preserves return addresses that point to memory regions which have become non-executable.

Implementation Overview

```
static int scan_thread(pid_t pid, pid_t tid, const map_t *maps, size_t nmaps, int verbose)
{
    // attach to target thread
    if (ptrace(PTRACE_ATTACH, tid, 0, 0) == -1) {
        return 0;
    }
    waitpid(tid, 0, 0);

    // initialize libunwind for remote unwinding
    unw_addr_space_t as = unw_create_addr_space(&_UPT_accessors, 0);
    void *ui = _UPT_create(tid);
    unw_cursor_t c;

    if (unw_init_remote(&c, as, ui) >= 0) {
        while (unw_step(&c) > 0 && frame < FRAME_CAP) {
            unw_word_t pc;
            unw_get_reg(&c, UNW_REG_IP, &pc);

            // find memory mapping for this address
```

```
            const map_t *m = find_map(maps, nmaps, (uintptr_t)pc);

            // check if memory is executable
            int exec = strchr(m->perms, 'x') != NULL;

            if (!exec) {
                // suspicious: return address points to non-executable memory
                suspicious_frames++;
            }
        }
    }
}
```

Understanding libunwind's Remote Stack Walking

libunwind implements architecture-independent stack unwinding by traversing the chain of stack frames in a target process. On x86-64, this process involves multiple unwinding methods depending on available information:

For code built by modern GCC/Clang, an .eh_frame section is always emitted even at -O2/-O3. libunwind decodes the Call-Frame-Information (CFI) opcodes to reconstruct all callee-saved registers and the previous stack pointer. Full debug-info sections like .debug_frame and .debug_info are not required—only the compact unwind tables in .eh_frame are needed.

If no valid CFI entry covers the current IP, libunwind falls back to chasing the canonical frame-pointer chain created by the prologue push rbp; mov rbp, rsp. The System V ABI frame layout stores the saved RBP of the caller at [RBP] and the return RIP at [RBP+8]. This fallback only works when the code was compiled with -fno-omit-frame-pointer or when individual functions maintain frame pointers (which varies by compiler optimisation and function complexity). However, for certain special frames like sigreturn and when both CFI and FP are unavailable, libunwind applies architecture-specific heuristics.

Detection in Action

Pendulum

Pendulum's multi-context chain creates numerous detectable artifacts:

```
[21:22:44] SCAN 1: PENDULUM DETECTED (ENCRYPTED STATE) - 1 suspicious threads
--- DETECTION DETAILS ---
    TID 334924 (pendulum):
        Frame 3 : PC 0x5691784c95ae  rw-p  NON-EXEC   [/tmp/pendulum/build/pendulum]
                   DEBUGGING NON-EXEC FRAME:
                   ├─ Memory region: 0x5691784c8000-0x5691784ca000 (8192 bytes)
                   ├─ Permissions: rw-p
                   ├─ Path: /tmp/pendulum/build/pendulum
                   ├─ Memory content at PC:
                   │   0x5691784c95ae: 0x15a7fc427c184b2b
                   │   0x5691784c95b2: 0xc39b953f15a7fc42
                   │   0x5691784c95b6: 0x1af0e2a7c39b953f
                   │   0x5691784c95ba: 0x1e0e74a11af0e2a7
                   ├─ Stack pointer (RSP): 0x7ffd57696300
                   ├─ Base pointer (RBP): 0x7ffd57696310
                   ├─ Stack content around RSP:
                   │   RSP-16: 0x7ffd57696310
                   │   RSP -8: 0x5691784c95ae ← MATCHES OUR PC!
                   │   RSP +0: 0x500000000
                   │   RSP +8: 0x569198aff2a0
                   │   RSP+16: 0x7ffd57696340
                   └─ ANALYSIS: Frame 3 = Likely return address from previous call
        Frame 4 : PC 0x5691784c9b42  rw-p  NON-EXEC   [/tmp/pendulum/build/pendulum]
                   DEBUGGING NON-EXEC FRAME:
                   ├─ Memory region: 0x5691784c8000-0x5691784ca000 (8192 bytes)
                   ├─ Permissions: rw-p
                   ├─ Path: /tmp/pendulum/build/pendulum
                   ├─ Memory content at PC:
                   │   0x5691784c9b42: 0xb339176d1e1e8370
                   │   0x5691784c9b46: 0x7c6e2ffdb339176d
                   │   0x5691784c9b4a: 0x5b4d6e537c6e2ffd
```

```
                    │   0x5691784c9b4e: 0xead6e8125b4d6e53
                    ├─ Stack pointer (RSP): 0x7ffd57696320
                    ├─ Base pointer (RBP): 0x7ffd57696340
                    ├─ Stack content around RSP:
                    │   RSP-16: 0x7ffd57696340
                    │   RSP -8: 0x5691784c9b42 ← MATCHES OUR PC!
                    │   RSP +0: 0x400000000
                    │   RSP +8: 0x569198aff2a0
                    │   RSP+16: 0x569198b0f600
                    └─ ANALYSIS: Frame 4 = Likely return address from previous call
        Frame 7 : PC 0x5691784c9105  rw-p  NON-EXEC   [/tmp/pendulum/build/pendulum]
                   DEBUGGING NON-EXEC FRAME:
                    ├─ Memory region: 0x5691784c8000-0x5691784ca000 (8192 bytes)
                    ├─ Permissions: rw-p
                    ├─ Path: /tmp/pendulum/build/pendulum
                    ├─ Memory content at PC:
                    │   0x5691784c9105: 0x42e1439dbd96652c
                    │   0x5691784c9109: 0xd16d9e8842e1439d
                    │   0x5691784c910d: 0xb371a19fd16d9e88
                    │   0x5691784c9111: 0x89b6e5fdb371a19f
                    ├─ Stack pointer (RSP): 0x7ffd57696450
                    ├─ Base pointer (RBP): (nil)
                    ├─ Stack content around RSP:
                    │   RSP-16: (nil)
                    │   RSP -8: 0x5691784c9105 ← MATCHES OUR PC!
                    │   RSP +0: 0x7ffd57696458
                    │   RSP +8: 0x38
                    │   RSP+16: 0x1
                    └─ ANALYSIS: Frame 7 = Likely return address from previous call
```

The detection reveals multiple non-executable frames in the call stack:

-

3 suspicious frames detected (Frames 3, 4, and 7)

- Each frame contains a return address pointing to rw-p memory (NON-EXEC)
- The frames contain return addresses that were pushed during context setup
- These addresses represent the control flow path through the context chain
- The return addresses point to locations that will be executed after decryption

The multiple frames correspond to Pendulum's context chain execution - as each context executes and transitions to the next, it leaves a return address on the stack. When the code becomes encrypted during sleep, these return addresses now point to non-executable memory, creating the detectable artifact. Ultimately, return addresses on the stack should **ALWAYS** point to executable memory.

SilentPulse

SilentPulse exhibits similar artifacts with different timing characteristics:

```
[17:10:55] SCAN 8: SilentPulse DETECTED (ENCRYPTED STATE) - 1 suspicious threads
--- DETECTION DETAILS ---
    TID 164018 (SilentPulse):
        Frame 2 : PC 0x5bddcf8512ba  rw-p  NON-EXEC   [/tmp/silentpulse/build/SilentPulse]
                   ├─ Memory region: 0x5bddcf850000-0x5bddcf852000 (8192 bytes)
                   ├─ Permissions: rw-p
                   ├─ Path: /tmp/silentpulse/build/SilentPulse
                   ├─ Stack content around RSP:
                   │   RSP -8: 0x5bddcf8512ba ← MATCHES OUR PC!
                   └─ ANALYSIS: Frame 2 = Likely return address from previous call
```

Notably, SilentPulse required multiple scan attempts before detection. This occurs because:

1. ptrace(PTRACE_ATTACH)
sends
SIGSTOP
to the target process
2.
This interrupts
epoll_wait()
with
EINTR
, causing immediate wake
3.
The process quickly decrypts, re-encrypts, and returns to sleep
4.
The detector must catch the brief window during context switching before re-encryption

Manual Analysis with GDB

While our automated detector uses libunwind to walk the call stack, we can also demonstrate a complementary detection method by manually examining the saved execution contexts with GDB. This approach directly inspects the heap-allocated ucontext structures rather than unwinding the stack, revealing the same fundamental issue from a different perspective.

During sleep, SilentPulse's .text section is encrypted and marked as non-executable (`rw-p`), but the saved execution context still contains return addresses pointing to this encrypted memory region.

This can be proven by examining a sleeping SilentPulse process:

```
$ ./SilentPulse
```

```
[DEBUG] .text @ 0x555555554000 - 0x555555555e8d (7821 bytes)
[DEBUG] sleeping for 5 seconds
```

Memory Protection During Sleep:

```
$ gdb -q -p $(pgrep SilentPulse)

(gdb) info proc mappings

# key Evidence - Memory Protection During Sleep:
Start Addr         End Addr           Perms  File
0x555555554000     0x555555555000     rw-p   /path/to/SilentPulse  <-- NON-EXECUTABLE
0x555555555000     0x555555556000     rw-p   /path/to/SilentPulse  <-- NON-EXECUTABLE
```

Finding the Problematic Address:

```
(gdb) break silentpulse_sleep
(gdb) break silentpulse.c:276
(gdb) continue

# examine the saved context after swapcontext
(gdb) print &state->ctx.main_context
$1 = (ucontext_t *) 0x5555555a7038

(gdb) x/50gx 0x5555555a7038
0x5555555a70d8: 0x00007fffffffd5f0   0x0000555555555e48
                                     ^^^^^^^^^^^^^^^^^^
                                     PROBLEMATIC ADDRESS!
```

Verification:

```
(gdb) info symbol 0x555555555e48
silentpulse_sleep + 413 in section .text

(gdb) x/5i 0x555555555e48
0x555555555e48 <silentpulse_sleep+413>: mov rax,QWORD PTR [rbp-0x58]
0x555555555e4c <silentpulse_sleep+417>: mov DWORD PTR [rax+0x48],0x0
0x555555555e53 <silentpulse_sleep+424>: lea rax,[rip+0x49f]
```

The manual inspection finds 0x555555555e48 stored in the heap-allocated ucontext_t structure:

1. Address Range: 0x555555555e48 falls within .text segment (0x555555554000-0x555555555e8d)
2. During Sleep: This .text region has rw-p permissions (non-executable)
3. Saved Context: The address is preserved in main_context.uc_mcontext.gregs[REG_RIP]
4. Detection: Return address points to non-executable memory = anomaly

The fundamental issue is that swapcontext() saves the current execution state into a heap-allocated structure. This saved state includes the instruction pointer that, when gets encrypted, now points to non-executable memory regions.

- Normal execution: Return addresses point to executable code (r-x)
- During sleep: Same addresses now point to encrypted regions in (rw-p) state = detection trigger

This demonstrates SilentPulse leaves forensic artifacts in multiple locations - both on the call stack (detected by libunwind) and in saved execution contexts (shown here with GDB)

Why Context-Based Obfuscation Creates Detectable Artifacts

The Artifact Generation Mechanism

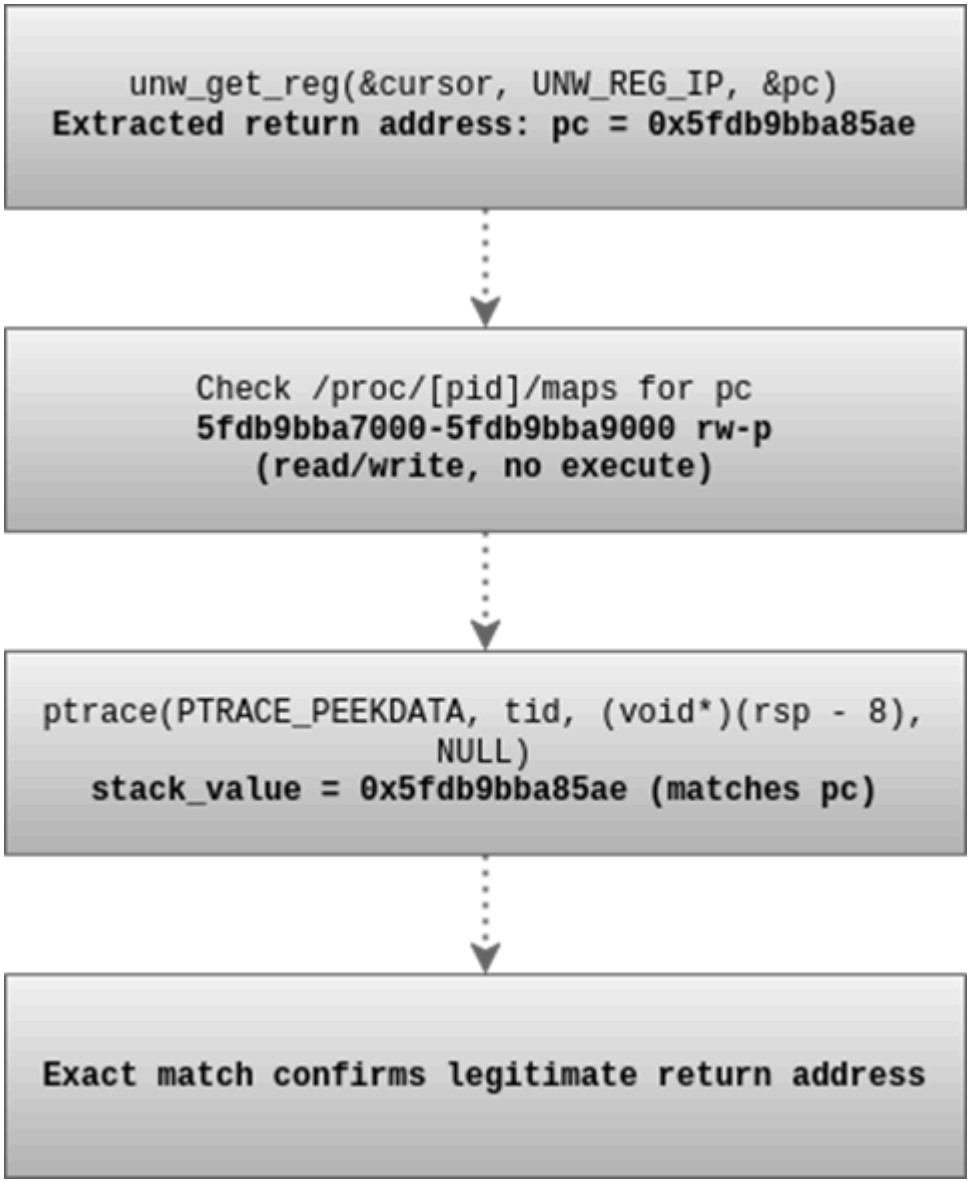
The detectable artifacts come from the normal function call mechanism. When the program calls functions like silentpulse_sleep(), the CPU pushes return addresses onto the stack:

1. Program execution flows through its functions normally
2. Each CALL instruction pushes a return address onto the stack
3. These return addresses point to locations in the program's .text segment
4. When encryption occurs, the .text segment becomes non-executable (rw-p)
5. The return addresses on the stack now point to non-executable memory

During the sleep phase:

- The .text segment is encrypted and protected as PROT_READ | PROT_WRITE
- Return addresses on the stack still point to these now non-executable locations
- libunwind walks the stack and finds these anomalous return addresses
- Detection occurs when return addresses point to non-executable memory

The detector leverages this state through systematic analysis:



This correlation proves:

•

The address was placed by a legitimate CALL instruction during program execution

•

It points to the process's main code segment (which should be executable)

Why These Artifacts Are Inherent to Standard Context-Based Implementations

Three architectural characteristics make these artifacts inherent to standard context-based approaches:

CPU Architecture Requirements: The x86-64 instruction set architecture mandates specific behaviour for function calls:

```
https://web.stanford.edu/class/cs107/guide/x86-64.html
"The callq instruction takes one operand, the address of the function
being called. It pushes the return address (current value of %rip,
which is the next instruction after the call) onto the stack and then
jumps to the address of the function being called."

In practice, when calling a function at 0x401234:
callq 0x401234
; CPU automatically executes:
; push %rip+5  (assuming 5-byte call instruction)
; jmp 0x401234
```

The RET instruction correspondingly pops and jumps to these addresses. No alternative exists for function return mechanisms.

Context Chain Dependencies: The execution flow must maintain:

- Return addresses on the stack from normal function calls
- The context chain linkage (via uc_link)
- The ability to resume normal execution after the sleep cycle

Temporal Exposure Window: Between encryption and decryption:

- The stack must remain intact to allow resumption
- Return addresses cannot be cleared without breaking execution flow
- Any stack manipulation would itself require executable code

This creates the following detection opportunity:

Before Encryption

Stack	Memory Permissions
0x7fff1234: 0x401234 **Valid Return Address**	.text segment (r-xp)
0x7fff123c: 0x401567 **__start_context**	.text segment (r-xp)
[RSP ->] Local Vars	

After Encryption (sleep)

Stack	Memory Permissions
0x7fff1234: 0x401234 **STILL VALID ADDRESS**	.text segment (rw-p)
0x7fff123c: 0x401567 **RETURN TO NON-EXEC > DETECTION TRIGGER**	.text segment (rw-p)
[RSP ->] Local Vars	

Detection: return address points to non-executable memory

Potential Defensive Approaches

While these artifacts are inherent to standard glibc-based context switching, various defensive techniques could theoretically complicate detection. For example, return address spoofing and call stack spoofing could be utilised, potentially evading stack-based detection. Also, function proxying mechanisms might establish intermediate trampolines in dynamically allocated executable memory, though such approaches would likely require additional execution threads, fundamentally altering the single-threaded design principle here.

Conclusion: Driving Linux Security Research Forward

The intent for this research with its mentioned limitations is geared toward one goal, to help accelerate the rate of innovation in evasion methods based upon Linux. This research is not just the innovative approach taken by SilentPulse, but also the detection techniques that determined it. There is already a strong technical foundation in Linux security research, but the commercialised red teaming community has a clear inclination towards Windows environments. However, publications such as tmpout have tackled this imbalance by creating dedicated platforms for Linux/ELF-focused security research.

As enterprise Linux deployments continue to grow, and threats target more and more edge systems and unmonitored enclaves, often running linux based operating systems. Defensive capabilities in these areas will mature, this means offensive capabilities focused on Linux will become more important and relevant. The aim is that research creates a positive feedback loop, where offensive improvements push defensive improvements, causing blue teams to create more advanced detection capabilities into a Linux context, building on what has been established already in enterprise windows security ecosystems. A large migration of government entities across Europe towards Linux and open-source alternatives is reshaping how enterprise attack surfaces are viewed. Schleswig-Holstein's move of 30,000 government workstations to Linux is a clear indication that this

movement is expanding, and the Danish Ministry of Digitalisation is further accelerating the process with a phased migration to Linux and LibreOffice, valuing digital sovereignty over cost. This shows that wider adoption of Linux based systems outside of just server workloads is growing, with some governments already starting to migrate. There are now regulated frameworks that support this approach, the 2024 Interoperable Europe Act requires EU governments to consider open-source first (and open-source technologies). This creates formal incentives to use these technologies, which in turn drives adoption. With the expansion of enterprise Linux environments, entities are now moving forward in a way that clearly requires modernised offensive research capabilities to actively mature the security improvement cycles.

This research gives insights to both communities. Red teams will learn about new techniques and their operational restrictions, and blue teams will have meaningful detection opportunities. More importantly, documenting these limitations provides clear guidance for future research to allow other researchers to combine our results with their own research or develop alternative tactics that do not have our limitations.

MEET THE AUTHOR

Keiran Mather Red Team Specialist

Keiran's role as a one of Bulletproof's Red Team members, sees him analysing and investigating all kinds of technology. You can find him writing about novel hacking techniques, exploits, and other security testing matters.

See other blogs by Keiran Mather

Follow Keiran Mather on LinkedIn